

STEAM MAIN GAME CHURN PROJECT

Grupo: Gabriel Cavalcante, Vitor Félix, Tharso Oliveira, João Paulo Costa

Disciplina: Ciência de Dados (FACAPE)

Professor: Mateus Amorim Silva

Data: 14/06/2026

1. INTRODUÇÃO

A indústria de jogos digitais movimenta bilhões de dólares anualmente e depende diretamente da capacidade de manter seus jogadores engajados por longos períodos. A perda de usuários ativos, fenômeno conhecido como churn, representa um desafio relevante para desenvolvedores e publicadoras, pois reduz a receita potencial, diminui a atividade das comunidades e compromete a longevidade dos jogos.

Identificar antecipadamente quais jogadores apresentam risco de abandonar um título permite que empresas adotem estratégias de retenção mais eficientes, como eventos personalizados, recompensas, campanhas promocionais ou atualizações direcionadas ao perfil dos usuários. Dessa forma, técnicas de Ciência de Dados e Aprendizado de Máquina podem ser utilizadas para detectar padrões comportamentais associados ao abandono antes que ele ocorra efetivamente.

Este trabalho tem como objetivo desenvolver um modelo de Machine Learning capaz de prever o abandono do jogo principal de usuários da plataforma Steam. Para isso, foi construído um dataset contendo informações relacionadas ao comportamento dos jogadores, como tempo total de jogo, quantidade de títulos na biblioteca, nível de atividade recente e concentração de horas no jogo mais utilizado. A partir desses dados, diferentes experimentos foram conduzidos utilizando o algoritmo Random Forest, comparando configurações distintas e avaliando o impacto de fatores como balanceamento de classes, seleção de atributos e data leakage. Por fim, o modelo selecionado foi disponibilizado por meio de uma API desenvolvida com FastAPI e containerizada com Docker, seguindo práticas modernas de MLOps e reprodutibilidade.

2. DADOS

Os dados utilizados neste trabalho foram obtidos a partir da plataforma Steam por meio de consultas, realizadas através da Steam API, a informações públicas dos perfis dos usuários. O processo de coleta teve como objetivo reunir métricas capazes de representar o comportamento dos jogadores em relação aos títulos presentes em suas bibliotecas, permitindo posteriormente a construção de um modelo preditivo de churn.

Após a etapa de coleta e tratamento, foi construído um dataset contendo aproximadamente 3.500 registros de usuários. Cada registro representa um jogador e suas respectivas características de utilização da plataforma e de seu jogo principal. Durante o desenvolvimento do projeto, registros com valores ausentes foram removidos para garantir a consistência dos dados utilizados no treinamento dos modelos.

As principais variáveis consideradas foram: *playtime_forever*, que representa o tempo total acumulado de jogo do usuário; *game_count*, correspondente à quantidade de jogos presentes na biblioteca; *recent_active_games*, que indica o número de títulos utilizados recentemente; *avg_playtime*, referente ao tempo médio de jogo por título; e *top_game_ratio*, que representa a proporção do tempo total dedicada ao jogo mais utilizado pelo jogador. Essas variáveis foram selecionadas por apresentarem potencial para capturar padrões de engajamento relacionados ao abandono de jogos.

Além dessas variáveis, durante os experimentos também foram analisadas as métricas *playtime_2weeks* e *engagement_ratio*. Entretanto, tais atributos participaram diretamente da construção do rótulo de churn durante a etapa de preparação dos dados. Por esse motivo, sua utilização em alguns modelos caracterizou um cenário de data leakage, empregado apenas para fins de comparação e demonstração dos impactos dessa prática sobre o desempenho dos algoritmos.

A variável alvo do problema foi denominada status. Inicialmente, o dataset foi estruturado como um problema de classificação multiclasse, contendo três categorias: *ativo*, *possível churn* e *churn*. Posteriormente, foram realizados experimentos adicionais convertendo o problema para classificação binária, agrupando a classe possível churn às demais categorias para avaliar diferentes estratégias de modelagem.

Em relação à distribuição das classes, observou-se um cenário moderadamente desbalanceado, com predominância de usuários classificados como churn. Para minimizar possíveis impactos desse desequilíbrio, foram realizados experimentos utilizando o parâmetro *class_weight="balanced"* do algoritmo Random

Forest, permitindo atribuir maior importância às classes menos representadas durante o treinamento.

Após as etapas de limpeza, transformação e validação, o dataset apresentou qualidade adequada para a construção dos modelos preditivos, possibilitando a realização dos experimentos descritos nas seções seguintes.

3. METODOLOGIA

O desenvolvimento deste projeto seguiu as etapas tradicionais de um fluxo de Ciência de Dados, abrangendo preparação dos dados, treinamento de modelos, avaliação de desempenho e disponibilização da solução em ambiente de produção.

Inicialmente, os dados coletados foram carregados e submetidos a um processo de limpeza, no qual registros contendo valores ausentes foram removidos. Em seguida, foram selecionadas as variáveis consideradas mais relevantes para a predição do abandono do jogo principal dos usuários.

Para a construção dos modelos foram utilizadas as variáveis *playtime_forever*, *game_count*, *recent_active_games*, *avg_playtime* e *top_game_ratio*. Essas características descrevem aspectos relacionados ao histórico de jogo, diversidade da biblioteca, atividade recente e grau de concentração de horas em um único título.

Após a seleção dos atributos, os dados foram divididos em conjuntos de treinamento e teste utilizando a estratégia holdout. Foi adotada uma proporção de 80% dos registros para treinamento e 20% para teste, mantendo a distribuição original das classes por meio da técnica de estratificação. Além disso, foi utilizado o valor *random_state* = 42 para garantir a reprodutibilidade dos experimentos.

O algoritmo escolhido para a construção dos modelos foi o Random Forest Classifier, disponível na biblioteca Scikit-Learn. A escolha desse algoritmo ocorreu devido à sua robustez em problemas de classificação, capacidade de lidar com relações não lineares entre variáveis e resistência a problemas de overfitting quando comparado a árvores de decisão individuais.

Diversos experimentos foram realizados ao longo do projeto com o objetivo de comparar diferentes configurações e estratégias de modelagem. Entre os experimentos desenvolvidos destacam-se:

- Modelo Random Forest sem data leakage;
- Modelo Random Forest com data leakage;
- Modelo sem balanceamento de classes;
- Modelo utilizando maior quantidade de árvores de decisão;

- Modelo utilizando conjunto reduzido de atributos;
- Modelos binários agrupando a classe possível churn às demais categorias;
- Modelo otimizado por Grid Search.

O experimento com data leakage foi utilizado para demonstrar os impactos da utilização de variáveis que participaram diretamente da construção do rótulo da variável alvo. Conforme esperado, esse cenário apresentou métricas artificialmente elevadas, evidenciando a importância de evitar vazamento de informação durante o treinamento de modelos preditivos.

Para rastrear e comparar os resultados dos experimentos foi utilizada a ferramenta MLflow. Cada execução registrou automaticamente parâmetros, métricas e artefatos produzidos durante o treinamento, permitindo a comparação sistemática entre diferentes versões dos modelos.

Posteriormente, foi aplicado o método Grid Search para otimização de hiperparâmetros do algoritmo Random Forest. Foram avaliadas diferentes combinações de parâmetros, incluindo número de árvores, profundidade máxima das árvores e quantidade mínima de amostras por divisão. O objetivo foi identificar automaticamente a configuração que apresentasse melhor desempenho durante a validação cruzada.

Após a seleção do modelo final, este foi serializado utilizando a biblioteca Joblib e disponibilizado por meio de uma API REST desenvolvida com FastAPI. A API permite receber informações de um jogador e retornar a previsão de sua categoria de churn de forma automática.

Por fim, a aplicação foi containerizada utilizando Docker, possibilitando a execução do sistema em diferentes ambientes computacionais sem necessidade de reconfiguração manual das dependências. Essa abordagem contribui para a reprodutibilidade do projeto e aproxima a solução das práticas adotadas em ambientes reais de Machine Learning Operations (MLOps).

4. RESULTADOS

Diversos experimentos foram conduzidos com o objetivo de avaliar o impacto de diferentes estratégias de modelagem sobre a capacidade de prever o abandono do jogo principal dos usuários. Os modelos foram comparados utilizando principalmente as métricas de acurácia, precisão, recall e F1-score.

O primeiro experimento utilizou um modelo Random Forest sem data leakage e considerando o problema original de classificação em três classes (ativo, possível churn e churn). Esse modelo obteve acurácia de aproximadamente 74,4%, apresentando desempenho satisfatório na identificação de usuários classificados

como churn, cujo F1-score foi de 0,87. Entretanto, a classe intermediária possível churn apresentou maior dificuldade de identificação, alcançando F1-score de apenas 0,20.

Posteriormente foi realizado um experimento contendo variáveis que participaram diretamente da construção da variável alvo. Nesse cenário ocorreu data leakage, resultando em métricas extremamente elevadas, incluindo acurácia próxima de 99,9% e F1-score praticamente perfeito para todas as classes. Embora os resultados sejam numericamente superiores, o modelo não representa uma solução válida para uso real, uma vez que parte da informação utilizada para prever o churn já estava implicitamente presente nas variáveis de entrada.

Com o objetivo de otimizar os hiperparâmetros do Random Forest, foi aplicada a técnica de Grid Search. O processo identificou como melhor configuração um modelo contendo 300 árvores de decisão e parâmetro *min_samples_split* igual a 5. Apesar da otimização, os ganhos observados foram modestos, elevando a acurácia de aproximadamente 74,4% para 74,7%. Esse resultado sugere que o modelo original já se encontrava próximo de uma configuração adequada para o problema.

Além dos modelos multiclasse, foram realizados experimentos convertendo o problema para classificação binária. No primeiro cenário, a classe possível churn foi agrupada à classe churn, representando todos os usuários com risco potencial de abandono. Esse modelo alcançou acurácia de aproximadamente 83,6%, além de F1-score de 0,90 para a classe churn.

No segundo cenário, a classe possível churn foi agrupada à classe ativo. O modelo resultante apresentou acurácia de aproximadamente 82,9% e F1-score de 0,87 para a classe churn. Embora os resultados também tenham sido satisfatórios, observou-se desempenho ligeiramente inferior ao obtido quando a classe possível churn foi incorporada à categoria churn.

A análise das importâncias das variáveis revelou um padrão consistente entre os experimentos. O atributo *recent_active_games* apresentou os maiores valores de importância em praticamente todos os modelos, indicando que o nível de atividade recente do usuário é o principal fator associado à previsão de abandono. Variáveis como *top_game_ratio* e *playtime_forever* também demonstraram contribuição relevante para o processo de classificação.

Considerando os resultados obtidos, o modelo binário que agrupou a classe possível churn à categoria churn foi selecionado como modelo final do projeto. Essa abordagem apresentou o melhor equilíbrio entre desempenho preditivo e aplicabilidade prática, permitindo identificar não apenas usuários que já abandonaram seu jogo principal, mas também aqueles que apresentam sinais iniciais de desengajamento e podem se tornar alvo de estratégias de retenção.

5. CONCLUSÃO

O presente trabalho teve como objetivo desenvolver uma solução baseada em Machine Learning para prever o abandono do jogo principal de usuários da plataforma Steam. A identificação antecipada de jogadores com risco de churn representa um recurso valioso para desenvolvedores e publicadoras, permitindo a implementação de estratégias de retenção mais eficientes e direcionadas.

Para atingir esse objetivo, foi construído um dataset contendo informações relacionadas ao comportamento dos usuários, seguido pela aplicação de técnicas de pré-processamento, engenharia de atributos e treinamento de modelos preditivos. O algoritmo Random Forest foi utilizado como principal abordagem de classificação, sendo submetido a diferentes experimentos para avaliar o impacto de fatores como data leakage, balanceamento de classes, otimização de hiperparâmetros e reformulação do problema para classificação binária.

Os resultados demonstraram que a presença de data leakage pode produzir métricas artificialmente elevadas, reforçando a importância de uma seleção cuidadosa das variáveis utilizadas durante o treinamento. Entre os modelos válidos avaliados, o melhor desempenho foi obtido na abordagem binária que agrupou a classe possível churn à categoria churn, alcançando aproximadamente 83,6% de acurácia e elevado desempenho na identificação de usuários em risco de abandono.

Além do desenvolvimento do modelo preditivo, o projeto incorporou práticas de MLOps por meio do uso do MLflow para rastreamento de experimentos, FastAPI para disponibilização do modelo como serviço e Docker para garantir portabilidade e reprodutibilidade da aplicação. Essas ferramentas permitiram aproximar a solução desenvolvida de um cenário real de produção.

Por fim, conclui-se que os objetivos propostos foram alcançados, resultando em um sistema capaz de auxiliar na identificação de jogadores com potencial de abandono de seu jogo principal. Como projetos futuros, podem ser explorados novos algoritmos de aprendizado de máquina, ampliação do conjunto de atributos utilizados e coleta de volumes maiores de dados, buscando aumentar ainda mais a capacidade preditiva do modelo.

6. REFERÊNCIAS

DOCKER INC. Docker Documentation. Disponível em: <https://docs.docker.com/>. Acesso em: 14 jun. 2026.

FASTAPI. FastAPI Documentation. Disponível em: <https://fastapi.tiangolo.com/>. Acesso em: 14 jun. 2026.

MLFLOW. MLflow Documentation. Disponível em: <https://mlflow.org/docs/latest/index.html>. Acesso em: 14 jun. 2026.

PANDAS DEVELOPMENT TEAM. Pandas Documentation. Disponível em: <https://pandas.pydata.org/docs/>. Acesso em: 14 jun. 2026.

PEDREGOSA, F. et al. Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, v. 12, p. 2825-2830, 2011. Disponível em: <https://jmlr.org/papers/v12/pedregosa11a.html>. Acesso em: 14 jun. 2026.

SCIKIT-LEARN. Scikit-learn Documentation. Disponível em: <https://scikit-learn.org/stable/>. Acesso em: 14 jun. 2026.

STEAMWORKS. Steam Web API Documentation. Disponível em: https://developer.valvesoftware.com/wiki/Steam_Web_API. Acesso em: 14 jun. 2026.

VALVE CORPORATION. Steam. Disponível em: <https://store.steampowered.com/>. Acesso em: 14 jun. 2026.